

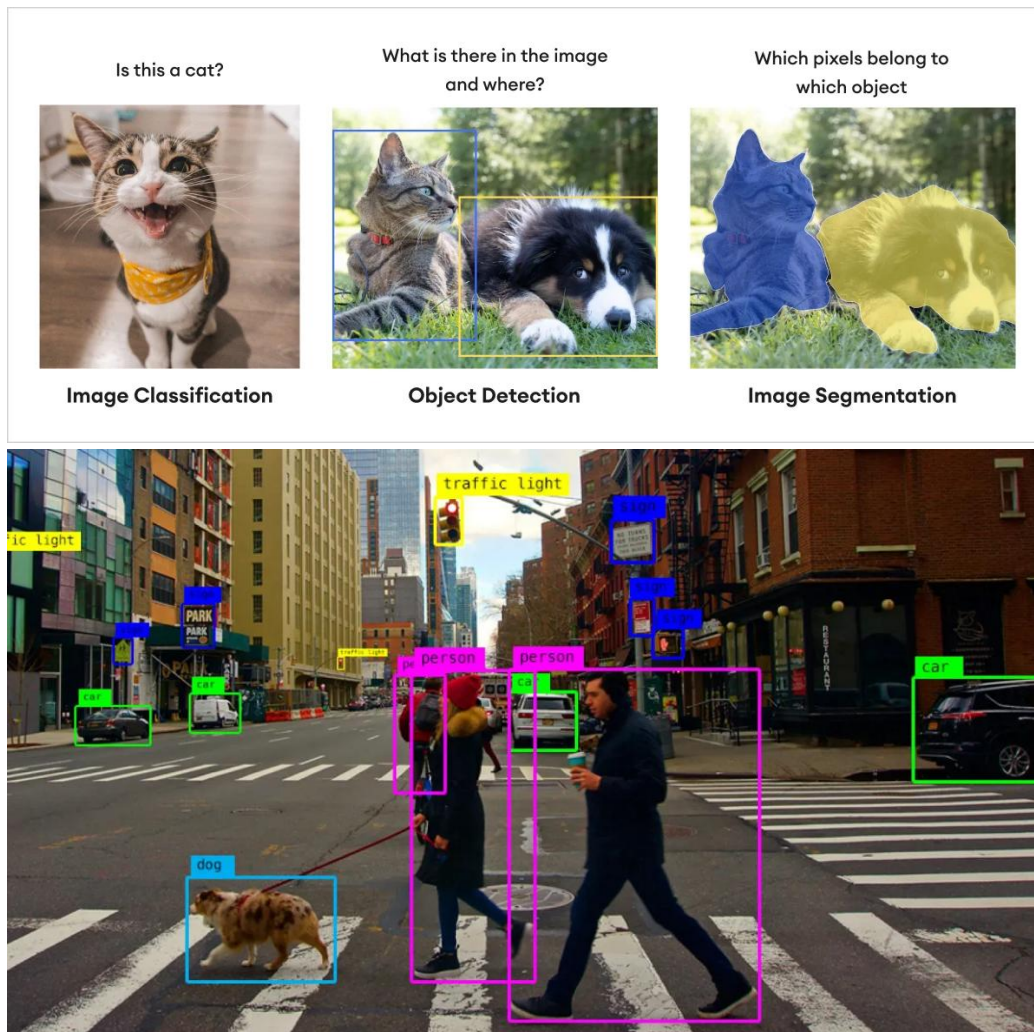
Chương 5: IMAGE RECOGNITION - OBJECT DETECTION **(PHÁT HIỆN ĐỐI TƯỢNG)**

Nội dung

1. Giới thiệu về Object Detection	2
2. Object Detection Model Training/Inference Pipeline – Luồng huấn luyện/suy luận)	4
3. Kiến trúc chung của một Mô hình (Pipeline)	5
4. Lịch sử phát triển	5
4.1. Họ R-CNN	6
4.2. Các mô hình One-Stage phổ biến	7
4.3. Họ YOLO (You Only Look Once).....	10
5. Lịch sử tiến hóa của Object Detection (Evolution Timeline).....	12
6. Loss Function – Hàm mất mát	13
6.1. Classification Loss - Hàm mất mát cho Phân loại.....	13
6.2. Bounding Box Regression Loss - Hàm mất mát cho Hồi quy Bounding box	14
7. Anchor trong Object Detection.....	16
8. Label Assignment - Gán nhãn	18
9. Metrics used for Evaluating Object Detection Models – Các chỉ số đánh giá mô hình	19
10. Những thách thức chính của Object Detection (Challenges)	21
11. So sánh Hiệu năng và Tốc độ thực tế.....	21
11.1. So sánh về Độ chính xác (AP@0.5)	21
11.2. So sánh về Tốc độ (Runtime - ms).....	22
11.3. Kết luận chọn mô hình	22

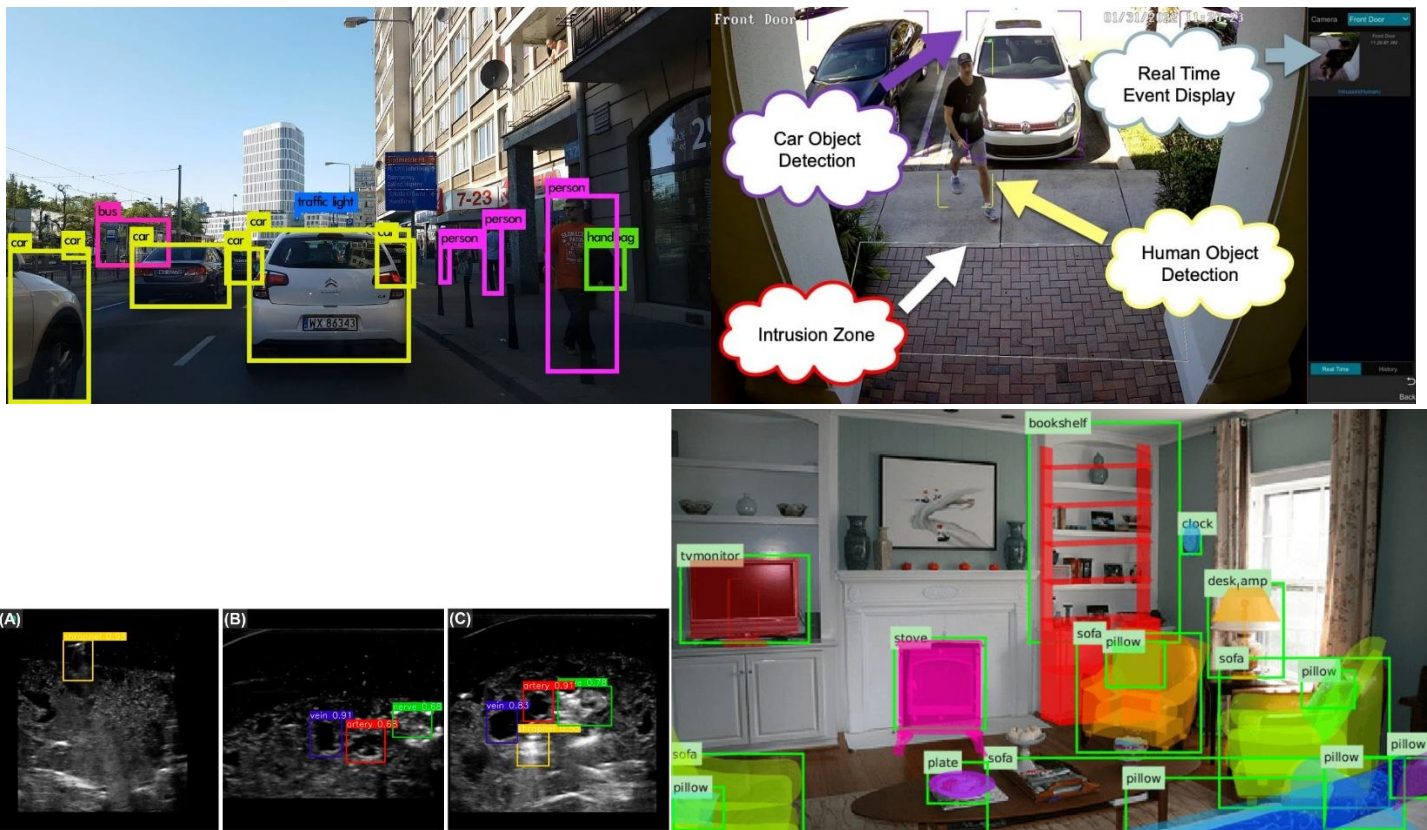
1. Giới thiệu về Object Detection

- Object detection is a computer vision technique that **identifies and localizes objects** within (trong) images or videos.
- Khác với bài toán **phân loại ảnh (Image Classification)** chỉ cho biết trong ảnh "có cái gì", **Object Detection** làm hai nhiệm vụ cùng lúc:
 - Định vị (Localization)**: Tìm vị trí đối tượng (thường là vẽ khung bao - bounding box).
 - Phân loại (Classification)**: Xác định đối tượng đó là gì (người, xe, chó, mèo...).
- Object detection not only **identifies objects but also determines their precise (chính xác) location** within the image **using bounding boxes**.
- Bounding boxes** are drawn around **objects of interest** in images.

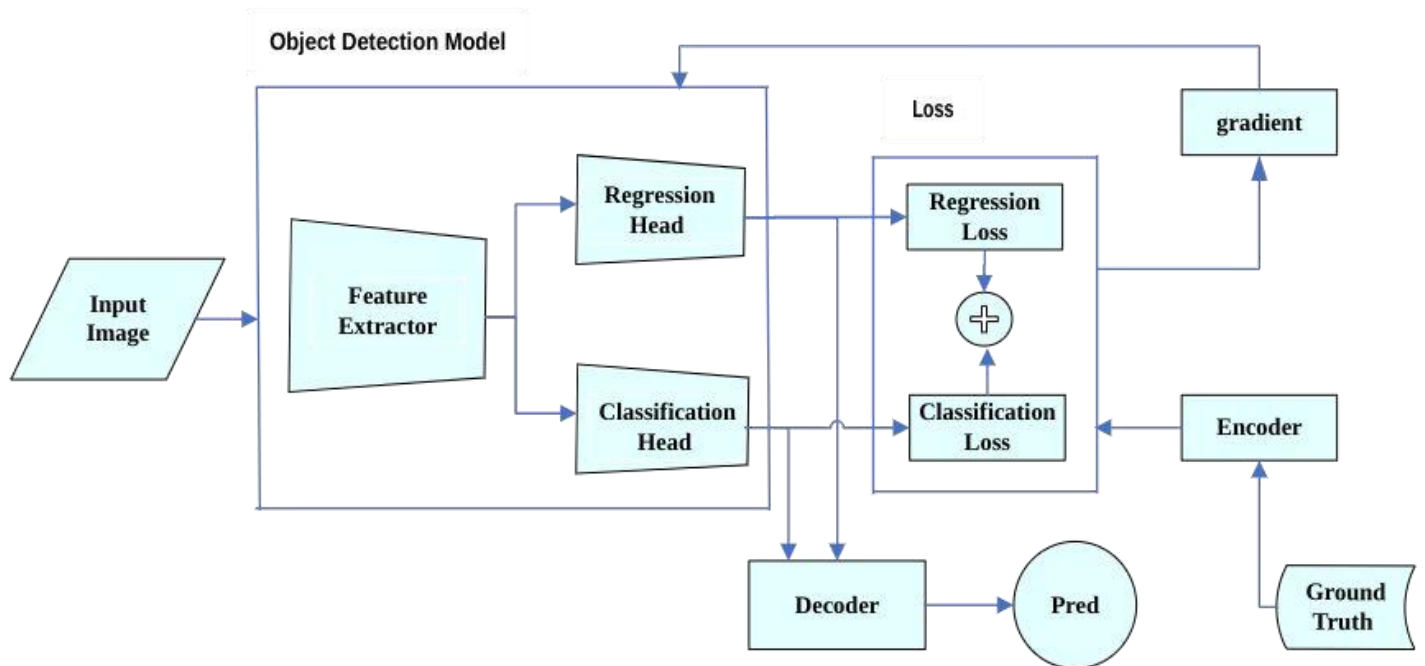


- Trong Object Detection, ta **không cần khôi phục lại ảnh gốc** (như trong bài toán Image Restoration) mà **chỉ trích xuất tọa độ và nhãn lớp**.

- Object Detection bao gồm **4 thành phần cơ bản**:
 - Object detection model** - Mô hình phát hiện đối tượng/Kiến trúc mạng nơ-ron
 - Ground truth encoding** - Mã hóa nhãn thực tế, cách mã hóa dữ liệu nhãn thật (để máy tính hiểu được tọa độ và lớp), ví dụ format YOLO .txt, COCO .json...
 - Loss function** - Hàm mất mát để đo sai số giữa dự đoán và thực tế.
 - Model prediction decoding** - Giải mã dự đoán của mô hình thành khung bao (bounding box) cụ thể trên ảnh.
- Ứng dụng**: tìm khuôn mặt, tìm vùng chứa chữ (OCR), tìm căn cước công dân, Autonomous vehicles, Surveillance and security (Giám sát và an ninh), Medical imaging, Augmented reality (AR - Thực tế tăng cường)...



2. Object Detection Model Training/Inference Pipeline – Luồng huấn luyện/suy luận)

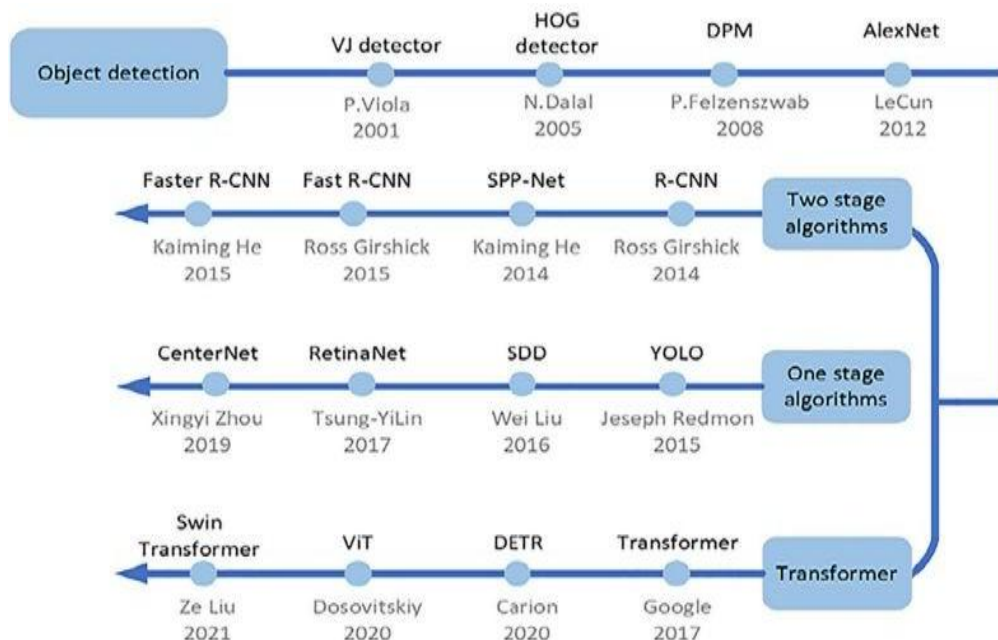


- Sơ đồ này minh họa kiến trúc "Head" tách biệt (**Decoupled Head**) thường thấy trong các mạng hiện đại như RetinaNet hay YOLOv6/v8, nơi việc xác định "ở đâu" và "là gì" được xử lý bởi hai nhánh riêng biệt sau khi đi qua Backbone.
- **Input Image:** Ảnh đầu vào.
- **Object Detection Model:**
 - **Feature Extractor (Backbone):** Trích xuất đặc trưng (ví dụ: mép, cạnh, hoa văn). Thường là ResNet, VGG, Darknet.
 - **Regression Head:** Nhánh dự đoán **tọa độ/vị trí khung bounding box** (x, y, w, h).
 - **Classification Head:** Nhánh dự đoán **xác suất lớp** (chó, mèo, xe...)/ Phân loại đối tượng.
- **Phần Training (Loss):**
 - Tính **Regression Loss** (sai số tọa độ) và **Classification Loss** (sai số phân loại).
 - Cộng 2 loss này lại -> Dùng **Gradient** để cập nhật trọng số (Backpropagation).
 - **Encoder/Ground Truth:** Dữ liệu thật được mã hóa để so sánh với dự đoán.
- **Phần Inference (Dưới cùng):**
 - **Decoder:** Giải mã output thô của mạng thành tọa độ thực tế.
 - **Pred:** Kết quả dự đoán cuối cùng.

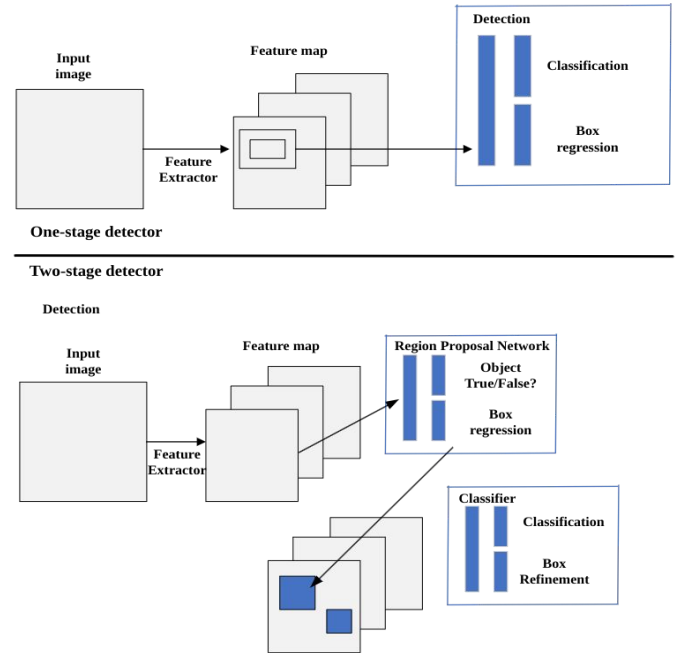
3. Kiến trúc chung của một Mô hình (Pipeline)

- Hầu hết các mô hình hiện đại (đặc biệt là dòng YOLO và One-stage) đều tuân theo luồng xử lý (pipeline) như sau:
 - Input Image:** Ảnh đầu vào được đưa vào mạng.
 - Backbone/ Xương sống (Feature Extractor):**
 - Nhiệm vụ:** Trích xuất đặc trưng (feature extraction) từ ảnh thô. Nó biến các pixel vô nghĩa thành các **feature maps** chứa thông tin về đường nét, hình dạng, texture.
 - Các loại phổ biến:** VGG, ResNet, MobileNet (cho thiết bị yếu), CSPNet (dùng trong YOLO) giúp giảm tính toán nhưng vẫn giữ được độ chính xác nhờ cơ chế chia luồng gradient, AlexNet, GoogleNet/Inception, EfficientNet
 - Neck/ Cổ:**
 - Nhiệm vụ:** Đây là phần quan trọng để phát hiện vật thể ở **nhiều kích thước khác nhau** (đa tỷ lệ). Nó trộn đặc trưng từ các tầng sâu (giàu ngữ nghĩa nhưng độ phân giải thấp) với các tầng nông (chi tiết không gian tốt).
 - Bộ phận trung gian thu thập đặc trưng từ Backbone và tạo ra các tháp đặc trưng (Feature Pyramids) giúp nhận diện vật thể ở nhiều kích thước khác nhau (ví dụ: FPN, PANet).
 - Head/ Đầu:** Phần cuối cùng chịu trách nhiệm đưa ra dự đoán cụ thể. Nó thường chia làm 2 nhánh:
 - Regression Head:** Dự đoán tọa độ khung bao.
 - Classification Head:** Dự đoán xác suất lớp của vật thể.

4. Lịch sử phát triển



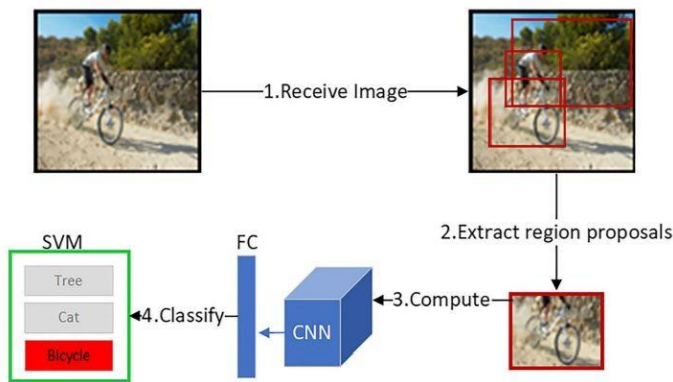
- VJ Detector (Viola-Jones - 2001, dùng cho Face Detection), HOG (2005), DPM. Các phương pháp này dựa trên **đặc trưng thủ công** (hand-crafted features).
- **AlexNet (2012)**: Sự bùng nổ của Deep Learning (CNN).
- **Two-stage (Hai giai đoạn)**: *R-CNN* -> *SPP-Net* -> *Fast R-CNN* -> *Faster R-CNN*. Đặc điểm: **Chính xác cao nhưng chậm**.
 1. Feature Extractor.
 2. **Region Proposal Network (RPN)**: **Đề xuất các vùng có thể chứa vật thể** (Object True/False?).
 3. **Refinement**: **Tinh chỉnh tọa độ/vị trí và phân loại đối tượng** trong các vùng đó.
 - *Ưu*: Chính xác cao.
 - *Nhược*: Chậm do 2 bước.
- **One-stage (Một giai đoạn)**: YOLO (2015), SSD, RetinaNet, CenterNet. Đặc điểm: **Nhanh, phù hợp realtime**.
 - Đi thẳng từ **Feature Map** -> **Detection**: Dự đoán trực tiếp tọa độ (bounding box) và lớp (class) mà không cần bước đề xuất vùng (RPN).
 - *Ưu*: Rất nhanh (Real-time), có thể thu được kết quả phát hiện trực tiếp.
 - *Nhược*: Trước đây kém chính xác hơn Two-stage (nhưng giờ YOLO đã bắt kịp).
- **Transformer (Mới nhất)**: DETR (Detection Transformer), ViT, Swin Transformer. Đây là **xu hướng hiện tại** (2020 trở đi).



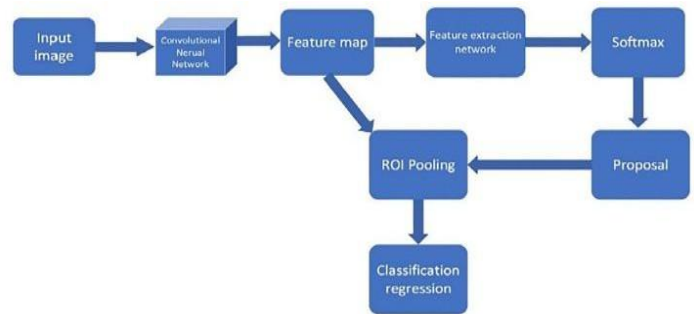
4.1. Họ R-CNN

- **R-CNN**: Dùng một thuật toán (Selective Search) để định vị **Roi (Region of Interest)** (cắt ra khoảng 2000 vùng nghi ngờ là vật thể trên ảnh). Sau đó đưa từng vùng một vào mạng CNN (DCNN) để phân loại.
 - **Nhược điểm**: Rất chậm vì phải chạy CNN hàng nghìn lần cho một bức ảnh.
- **SPPNet & Fast R-CNN**: Cải thiện tốc độ bằng cách trích xuất Roi từ feature maps
 - Thay vì cắt ảnh gốc, nó đưa cả ảnh vào mạng CNN để lấy ra một "bản đồ đặc trưng" (feature map) chung. Sau đó mới cắt vùng (Roi) trên bản đồ đặc trưng này.
 - **Kết quả**: Nhanh hơn nhiều so với R-CNN truyền thống vì chỉ cần chạy CNN một lần.
- **Faster R-CNN**: Sử dụng RPN (Region Proposal Network) để tạo anchor boxes
 - Loại bỏ thuật toán Selective Search thủ công. Nó giới thiệu **RPN (Region Proposal Network)** - một mạng nơ-ron nhỏ chuyên nhiệm vụ **tự động đề xuất các vùng chứa vật thể** (dựa trên các hộp neo - anchor boxes).
 - **Kết quả**: Mô hình được huấn luyện "end-to-end" (đầu cuối), nhanh và chính xác hơn.
- **Mask R-CNN**: Thêm nhánh dự đoán mask cho segmentation, phát triển từ Faster R-CNN. Không chỉ vẽ hình chữ nhật bao quanh vật thể, nó còn tô màu chính xác từng pixel của vật thể đó (Segmentation).

- **R-FCN:** Thay thế fully connected layers bằng position-sensitive score maps (bản đồ điểm nhạy cảm vị trí) để phát hiện vật thể tốt hơn và nhẹ hơn.
- **Cascade R-CNN:** Huấn luyện chuỗi detectors với ngưỡng IoU tăng dần.
 - **Vấn đề:** Khi huấn luyện và khi dùng thực tế, tiêu chuẩn đánh giá độ khớp (IoU) có thể bị lệch.
 - **Giải pháp:** Sử dụng một chuỗi (cascade) **các bộ phát hiện nối tiếp nhau**. Bộ sau có tiêu chuẩn khắt khe hơn bộ trước, giúp lọc dần để kết quả cuối cùng cực kỳ chính xác.
 - Chống quá khớp – Overfitting



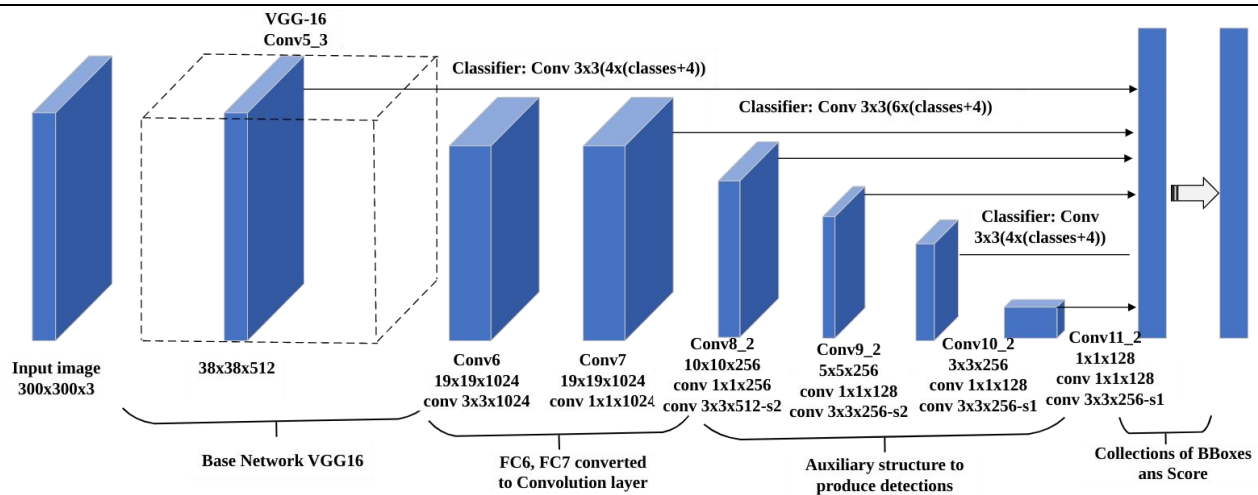
The four steps of R-CNN



Detection process of Faster R-CNN

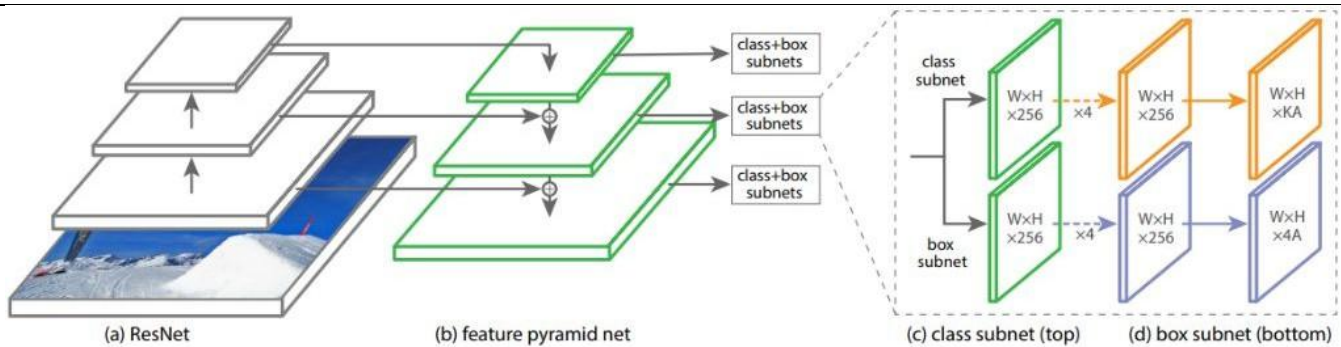
4.2. Các mô hình One-Stage phổ biến

- **SSD (Single Shot Detector)**
 - **Ý tưởng cốt lõi:** "**Nhìn một lần là ra ngay**" (Single Shot). SSD đặt dày đặc các hộp neo (anchor boxes) dày đặc lên ảnh đầu vào và dự đoán vật thể ngay lập tức mà **không cần bước đề xuất vùng** (như R-CNN).
 - **Kiến trúc (VGG-16 Backbone):** SSD gồm 2 phần chính: Mạng nền và Phần đầu SSD (SSD Head).
 - **Mạng nền (Base Network - VGG16):**
 - Sử dụng mạng VGG-16 đã được huấn luyện trước (pre-trained).
 - Các lớp Fully Connected (FC6, FC7) được chuyển đổi thành các lớp Convolution để giữ nguyên không gian hình ảnh.
 - **Cấu trúc kim tự tháp (Auxiliary Structure):**
 - Đây là điểm hay nhất của SSD. Sau phần mạng nền, **nó thêm vào các lớp Conv8_2, Conv9_2, Conv10_2, v.v., với kích thước giảm dần** (từ to đến nhỏ).
 - **Nguyên lý hoạt động:**
 - **Các lớp đầu** (kích thước lớn, ví dụ 38x38): Dùng để phát hiện các vật thể **nhỏ**.
 - **Các lớp sau** (kích thước nhỏ, ví dụ 1x1): Dùng để phát hiện các vật thể **lớn**.
 - Mỗi lớp này đều có khả năng đưa ra dự đoán (Classify) và tinh chỉnh hộp (Regress) độc lập.



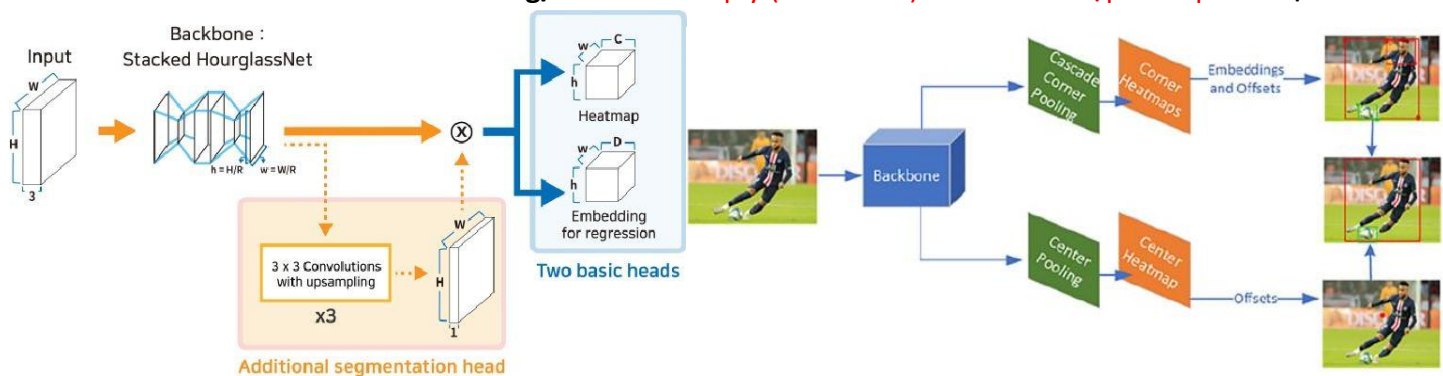
• RetinaNet

- **Ý tưởng cốt lõi:** Giải quyết vấn đề mất cân bằng dữ liệu để mô hình một giai đoạn (one-stage) có thể chính xác ngang ngửa mô hình hai giai đoạn. Đặc biệt tốt cho vật thể nhỏ và dày đặc.
- **Cải tiến chính:**
 - **Feature Pyramid Networks (FPN)** - Mạng kim tự tháp đặc trưng
 - **Focal Loss** - Tập trung vào các mẫu khó
- **Thành phần:**
 - **Bottom-up pathway (Backbone: ResNet):** Nó tính toán các bản đồ đặc trưng ở các tỷ lệ khác nhau. Càng đi sâu, hình ảnh càng nhỏ lại nhưng thông tin càng mang tính trừu tượng cao.
 - **Top-down pathway + Lateral connections (Đường đi từ trên xuống & Kết nối ngang)**
 - Mạng lấy các đặc trưng mạnh (từ lớp sâu/nhỏ) phóng to lên (upsample) và cộng gộp với các đặc trưng chi tiết (từ lớp nông/lớn) thông qua kết nối ngang.
 - **Mục đích:** Tạo ra một tháp đặc trưng (Pyramid) mà ở tầng nào cũng vừa có ngữ nghĩa mạnh, vừa có vị trí chính xác.
 - **Classification subnetwork (Nhánh phân loại):** Tại mỗi vị trí trên lưới, nhánh này dự đoán xác suất xem "có vật thể không và là vật thể gì?" (con chó, cái xe...).
 - **Regression subnetwork (Nhánh hồi quy):** Song song với nhánh phân loại, nhánh này tính toán độ lệch (offset) để nắn chỉnh cái hộp neo (anchor box) sao cho khít nhất với vật thể thật.
- **Điểm đặc biệt: Focal Loss**
 - Bình thường, trong ảnh có quá nhiều nền (bầu trời, đường phố...) dễ nhận biết, làm lấn át các vật thể khó.
 - **Focal Loss** giúp mô hình "bớt quan tâm" đến những thứ dễ (nền) và "tập trung cao độ" vào những mẫu khó (vật thể nhỏ, bị che khuất).
- **Ưu điểm:** Hiệu quả với đối tượng dày đặc và nhỏ.



CenterNet

- **Ý tưởng cốt lõi:** Thay vì vẽ rất nhiều "hộp" (anchor boxes) rồi chọn cái tốt nhất như các mô hình khác, CenterNet coi vật thể chỉ là **một điểm duy nhất (điểm trung tâm - center point)**.
- **Cách thức hoạt động:**
 - **Dự đoán điểm:** Mô hình xác định vật thể bằng cách tìm tọa độ (x, y) của tâm vật thể đó → Dự đoán đối tượng như **single points** (điểm đơn).
 - **Dự đoán kích thước:** Đồng thời, nó dự đoán chiều rộng và chiều cao (vùng bao phủ) của vật thể từ tâm đó (diện tích).
- **Hiệu quả:** Đây là kỹ thuật độc đáo đã được chứng minh là vượt trội hơn các biến thể như SSD hay dòng R-CNN về sự cân bằng giữa tốc độ và độ chính xác.
- **Kiến trúc chi tiết (Stacked HourglassNet):**
 - **Đầu vào (Input):** Hình ảnh gốc.
 - **Backbone (Mạng nền):** Sử dụng kiến trúc "Stacked Hourglass" (Đồng hồ cát xếp chồng).
 - Tại sao gọi là Đồng hồ cát? Mạng này liên tục thu nhỏ ảnh (downsample) để lấy đặc điểm ngữ nghĩa, rồi lại phóng to ảnh (upsample) để lấy lại chi tiết vị trí, tạo hình dáng giống cái đồng hồ cát.
 - Quá trình này giúp mạng nhìn thấy cả chi tiết nhỏ lẫn ngữ cảnh lớn.
 - **Các nhánh đầu ra (Heads):** Từ backbone, mạng chia ra 2 nhánh chính:
 - **Heatmap:** Để tìm vị trí tâm vật thể (phân loại xem đó là cái gì).
 - **Embedding/Size:** Để hồi quy (tính chính) kích thước hộp bao quanh vật thể.



4.3. Họ YOLO (You Only Look Once)

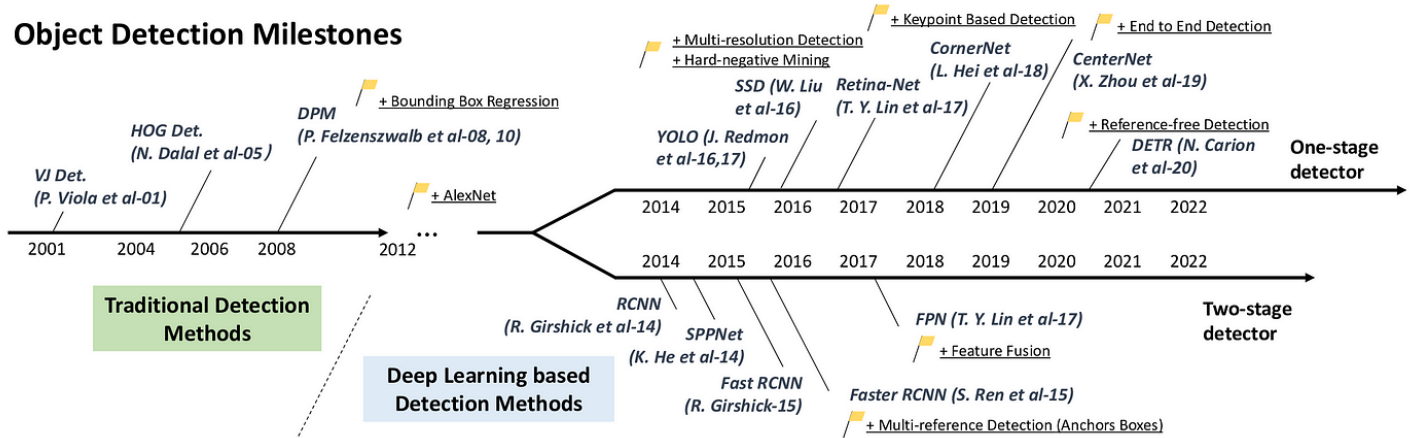
- Là dòng mô hình one-stage nổi tiếng nhất vì cân bằng tuyệt vời giữa tốc độ và độ chính xác.
- **YOLOv1**
 - Chia ảnh thành lưới $S \times S$
 - Sử dụng ít anchor boxes hơn để hồi quy và phân loại.
 - Backbone: Darknet
 - Nhanh nhưng chưa chính xác cao.
- **YOLOv2**
 - Cải thiện bằng nhiều anchor boxes hơn và phương pháp hồi quy hộp bao mới.
 - Backbone: Darknet-19 (**19 conv layers** + 5 max pooling)
- **YOLOv3**
 - Mạng sâu hơn với thay đổi nhỏ
 - Backbone: Darknet-53 (mô hình Residual sâu hơn)
 - Thời gian inference/suy luận cực nhanh, ~30ms/ảnh.
 - Phần Neck sử dụng **FPN** (Feature Pyramid Network) giống RetinaNet.
- **YOLOv4**
 - Cải tiến nhiều mặt so với v3: data processing, training, **activation**, **loss function**
 - **Backbone**: Nâng cấp lên **CSPDarknet53**.
 - **Neck**: Kết hợp **SPP** (Spatial Pyramid Pooling) và **PANet** (Path Aggregation Network) để tổng hợp đặc trưng tốt hơn.
- **YOLOv5**
 - Cải tiến từ YOLOv4 với **mosaic augmentation** technique (ghép 4 ảnh làm 1 khi huấn luyện) để tăng hiệu suất tổng quát.
 - **Backbone**: CSPDarknet53 kết hợp cấu trúc Focus.
 - **Neck**: Sử dụng PANet.

Layers	YOLOv3	YOLOv4	YOLOv5
Neural Network Type	FCNN	FCNN	FCNN
Backbone	Darknet53	CSPDarknet53 (CSPNet in Darknet)	CSPDarknet53 Focus structure
Neck	FPN (Feature Pyramid Network)	SPP (Spatial Pyramid Pooling) and PANet (Path Aggregation Network)	PANet
Head	B x (5 + C) output layer B : No. of bounding boxes C : Class score	Same as Yolo v3	Same as Yolo v3
Loss Function	Binary Cross Entropy	Binary Cross Entropy	Binary Cross Entropy and Logit Loss Function

- **FCNN** là viết tắt của **Fully Convolutional Neural Network** (Mạng nơ-ron tích chập toàn phần). Các mạng đời cũ (như YOLOv1) thường có lớp cuối cùng là lớp **Kết nối đầy đủ (Fully Connected - FC)**, lớp này đòi hỏi ảnh đầu vào phải có kích thước cố định (ví dụ 448x448).
- **FCNN**: Loại bỏ lớp FC này và thay thế hoàn toàn bằng các lớp Tích chập (Convolution).
- **Lợi ích**: Giúp **mạng có thể nhận đầu vào là ảnh có bất kỳ kích thước nào** mà không bị lỗi, đồng thời **giảm lượng tham số tính toán** giúp mô hình nhẹ và nhanh hơn.
- **YOLOv6** do Meituan (một tập đoàn công nghệ Trung Quốc) phát triển, tập trung vào ứng dụng công nghiệp.
- **YOLOv7** do nhóm tác giả của YOLOv4 (WongKinYiu) phát triển, được đánh giá rất cao về mặt học thuật.
 - *Vì v5 và v8 cùng "một mẹ" (Ultralytics) và có hệ sinh thái phần mềm rất mạnh nên nhiều tài liệu thường gom nhóm chúng lại với nhau, vô tình bỏ qua v6 và v7.*
- **YOLOv8**
 - Hỗ trợ: **object detection, classification, instance segmentation**
 - Phát triển bởi Ultralytics, tập trung vào cải tiến kiến trúc và trải nghiệm cho lập trình viên.
- **YOLOv9 (2024)**: Giới thiệu Thông tin độ dốc có thể lập trình (Programmable Gradient Information - PGI) và kiến trúc GELAN để nâng cao hiệu suất.
- **YOLOv10 (2024)**: Tập trung vào đào tạo không NMS (Non-Maximum Suppression) và thiết kế thân thiện với phần cứng để ứng dụng trong AI biên (edge AI).
- **YOLOv11 (2025)**: Sử dụng mô hình kết hợp CNN-transformer, mang lại hiệu suất cao trên nhiều tác vụ và tối ưu cho các thiết bị biên.
- **YOLOv12 (2025)**: Phiên bản mới nhất, giới thiệu kiến trúc tập trung vào cơ chế chú ý (attention-centric architecture) và các cơ chế chú ý vùng hiệu quả (Area Attention - A2) để cải thiện độ chính xác và sự ổn định khi huấn luyện.
- **YOLOv13**: Được phát triển bởi iMoonLab và công bố vào giữa năm 2025. Mô hình này giới thiệu cơ chế Tăng cường Tương quan Thích ứng dựa trên Siêu đồ thị (Hypergraph-based Adaptive Correlation Enhancement - HyperACE) để hiểu các mối quan hệ phức tạp giữa các đối tượng trong cảnh. Nó cũng sử dụng mô hình Phân phối và Tổng hợp Toàn bộ Quy trình (Full-Pipeline Aggregation-and-Distribution - FullPAD) để chia sẻ tính năng hiệu quả hơn.
- **YOLO26 (2025)**: Next-gen model, further optimizing for edge AI deployment.
- **Cấu trúc YOLO gồm 3 thành phần chính:**
 - **Backbone** - Trích xuất đặc trưng chính, and feeds them to the Head through Neck.
 - **Neck** - Thu thập feature maps (extracted by the Backbone), và tạo feature pyramids
 - **Head** - Output layers

5. Lịch sử tiến hóa của Object Detection (Evolution Timeline)

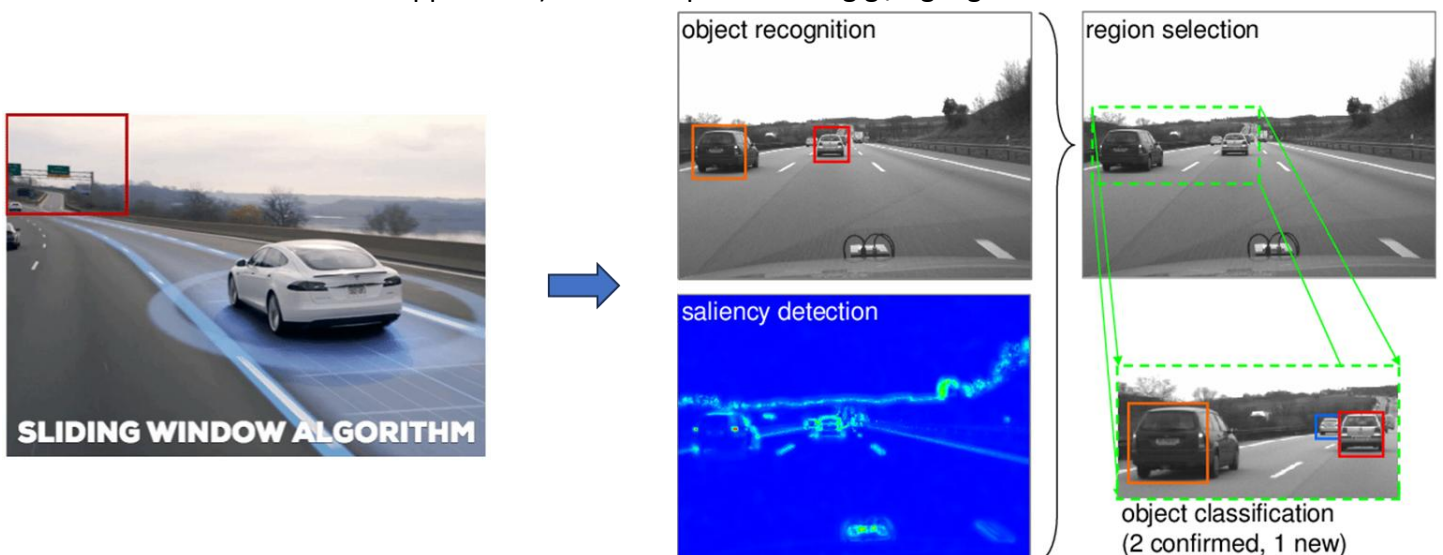
Object Detection Milestones



Sự phát triển chia làm 2 kỷ nguyên rõ rệt:

• Phương pháp truyền thống (Traditional Pipeline - Trước 2012):

- Phương pháp cổ điển, dựa nhiều vào việc xử lý thủ công.
- **Quy trình:** Tiền xử lý → Cắt vùng (Sliding Window) → Trích xuất đặc trưng thủ công (HOG, SIFT) → Phân loại (SVM).
- **Đặc điểm:** Chậm, phụ thuộc vào việc con người chọn đặc trưng "bằng tay".
- Trước khi có AI hiện đại, quy trình phát hiện vật thể gồm 4 bước cứng nhắc:
 1. **Tiền xử lý (Pre-processing):** Làm sạch ảnh, chuẩn hóa kích thước, tăng độ tương phản...
 2. **Trích xuất vùng quan tâm (ROI Extraction):** Sử dụng thuật toán **Cửa sổ trượt (Sliding Window)**.
 Cách hoạt động: Một ô cửa sổ hình chữ nhật sẽ trượt khắp bức ảnh từ trái qua phải, từ trên xuống dưới. Tại mỗi vị trí, nó cắt ra một phần ảnh để kiểm tra xem "có gì trong này không?".
 3. **Phân loại vật thể (Object classification):** Phần ảnh cắt ra ở bước trên được đưa vào bộ phân loại (như SVM) để **định danh** vật thể.
 4. **Tinh chỉnh (Refinement):** Kết hợp các kết quả, loại bỏ các hộp trùng lặp (thường dùng Non-Maximum Suppression) để ra kết quả cuối cùng gọn gàng nhất.

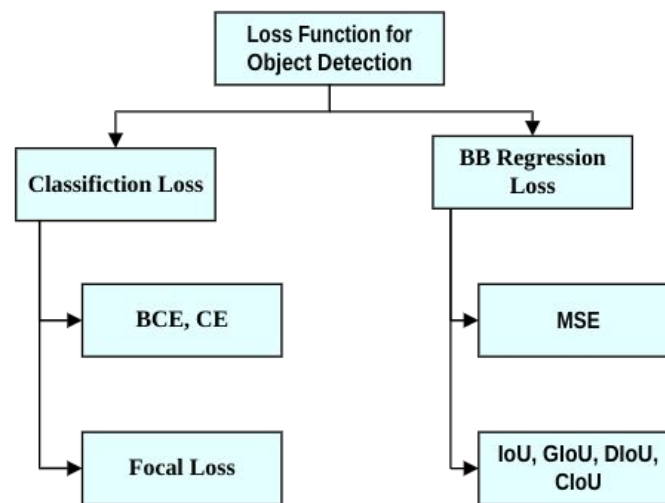


- **Kỷ nguyên Deep Learning (Sau 2012):**

- Phương pháp hiện đại, mô hình tự học đặc trưng.
- **Bước ngoặt:** AlexNet (2012) chứng minh sức mạnh của mạng nơ-ron tích chập (CNN).
- **Phân nhánh:**
 - **Two-stage (Hai giai đoạn):** Chính xác nhưng chậm (R-CNN, Fast R-CNN, Faster R-CNN).
 - **One-stage (Một giai đoạn):** Nhanh, thời gian thực (YOLO, SSD, RetinaNet).
 - **Anchor-free & Transformer:** Xu hướng mới (CenterNet, DETR) loại bỏ sự phụ thuộc vào Anchor box.

6. Loss Function – Hàm mất mát

- Hàm mất mát giống như "người thầy" chấm điểm cho mô hình.
- Điểm càng thấp (Loss thấp), mô hình học càng giỏi.



6.1. Classification Loss - Hàm mất mát cho Phân loại

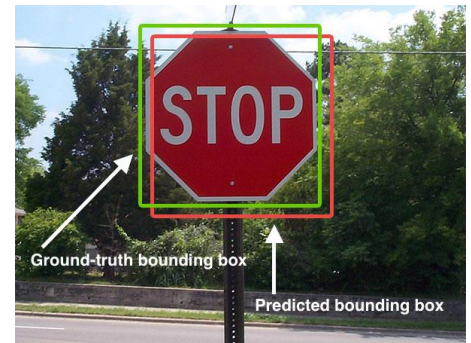
- Vấn đề lớn nhất trong Object Detection là **Mất cân bằng lớp (Class Imbalance)**: Trong một ảnh, số lượng vị trí "nền" (không có gì) luôn áp đảo số lượng vị trí có vật thể.
- **Cross-Entropy (CE) Loss**
 - **Mục đích:** Đánh giá việc mô hình phân loại đúng hay sai một cách cơ bản.
 - **Nhược điểm:** Nó đối xử công bằng với mọi ví dụ. Vì số lượng "nền" quá lớn, tổng lỗi từ các vùng nền dễ dàng áp đảo lỗi từ các vật thể hiếm, làm mô hình bị thiên lệch (bias), lười học cái khó.
 - **Công thức:** $CE(p_i) = -\log(p_i)$, với p_i (Probability): **Xác suất mô hình dự đoán đúng**. Ví dụ vật thể thật là "Chó", mô hình đoán là "Chó" với độ tin cậy 0.8 thì $p_i = 0.8$.

- **Focal Loss (FL)**

- Mục đích: Bắt mô hình tập trung vào các ca khó (Hard examples).
- Công thức: $FL(p_i) = -\alpha(1-p_i)^\gamma \log(p_i)$
 - α (Alpha): **Hệ số cân bằng lớp**. Giúp tăng tầm quan trọng của các lớp hiếm (vật thể) và giảm tầm quan trọng của lớp phổ biến (nền). Thường $\alpha \in [0,1]$.
 - γ (Gamma): **Tham số tập trung** (Focusing parameter). Đây là "ngôi sao" của công thức.
 - Nếu p_i **lớn (mẫu dễ)**: $(1-p_i)$ sẽ nhỏ $\rightarrow$ Loss giảm mạnh gần 0 $\rightarrow$ Mô hình ít quan tâm.
 - Nếu p_i **nhỏ (mẫu khó)**: $(1-p_i)$ sẽ gần bằng 1 $\rightarrow$ Loss cao/ giữ nguyên $\rightarrow$ Mô hình tập trung học.
- Ý nghĩa thực tiễn:
 - Trong ảnh có rất nhiều nền trời (dễ nhận biết). Nếu dùng hàm thường, mô hình sẽ thỏa mãn vì đoán đúng nền trời.
 - Focal Loss sẽ giảm điểm số của phần dễ này xuống gần bằng 0, ép mô hình phải tìm cách đoán đúng những vật thể nhỏ, khó nhìn thì mới giảm được Loss tổng.

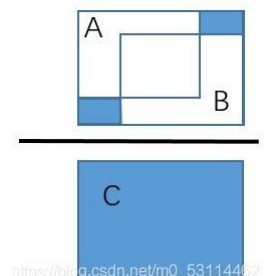
6.2. Bounding Box Regression Loss - Hàm mất mát cho Hồi quy Bounding box

- Để đo xem cái hộp mô hình vẽ ra lệch bao nhiêu so với hộp thật.
- **Intersection over Union (IoU)**:
 - Mục đích: Thước đo cơ bản nhất xem 2 hình chữ nhật chồng lên nhau bao nhiêu.
 - Công thức: $IoU = (A \cap B) / (A \cup B)$
 - If $IoU \geq \text{threshold}$ $\rightarrow$ the predicted box = true positive.
 - If $IoU < \text{threshold}$ $\rightarrow$ the predicted box = false positive.
 - If a true object is not detected $\rightarrow$ the ground truth label has a false negative.
 - Vấn đề: Nếu 2 hộp nằm xa nhau, không chạm tí nào $\rightarrow IoU = 0$. Lúc này đạo hàm bằng 0, mô hình không biết phải di chuyển hộp dự đoán theo hướng nào để chạm vào hộp thật.



- **GIoU (Generalized IoU)**:

- Giải quyết vấn đề của IoU khi 2 hộp không chạm nhau tí nào, $IoU = 0$.
- Công thức: $L_{GIoU} = 1 - IoU + \frac{|C \setminus (A \cup B)|}{|C|}$
- Tham số:
 - C : Hộp bao nhỏ nhất (Convex Hull) chứa cả hộp A và hộp B .
 - $|C|$: Diện tích của hộp C .
 - $|C \setminus (A \cup B)|$: Phần diện tích "thừa" trong hộp C (không chứa A hay B). Mục tiêu là giảm thiểu phần thừa này để ép 2 hộp lại gần nhau.



- **Ý nghĩa:** Nó vẽ ra một hộp bao lớn (C) bao trùm cả 2. Nó phạt mô hình nếu để khoảng trống trong hộp bao này quá nhiều. Điều này ép hộp dự đoán phải di chuyển dần về phía hộp thật ngay cả khi chưa chạm nhau.

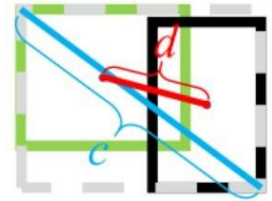
- **DIoU (Distance IoU):**

- **Cải tiến:** Tính trực tiếp khoảng cách giữa tâm của 2 hộp, giúp hội tụ nhanh hơn.

- **Công thức:** $L_{DloU} = 1 - IoU + \frac{\rho^2(b, b^{gt})}{c^2}$

- **Tham số:**

- b : Điểm tâm của hộp dự đoán.
- b^{gt} : Điểm tâm của hộp thật ($gt=Ground Truth$).
- ρ (**rho**): Khoảng cách Euclid (đo độ dài đoạn thẳng nối 2 tâm). $d = \rho(b, b^{gt})$
- c : Độ dài đường chéo của hộp bao nhỏ nhất chứa cả 2 hộp.



- **Ý nghĩa:** Thay vì chỉ cố gắng làm tăng diện tích chồng lấn (đôi khi rất chậm), DIoU trực tiếp kéo tâm của hộp dự đoán về trùng với tâm hộp thật. Giống như việc ngắm bắn thẳng vào hồng tâm, giúp mô hình học nhanh hơn và ổn định hơn.

- **CIoU (Complete IoU):**

- **Toàn diện nhất:** Tính thêm cả sự sai lệch về tỷ lệ khung hình (dài/rộng).
- Đây là hàm loss hiện đại thường dùng trong YOLO

- **Công thức:** $L_{CIoU} = 1 - IoU + \frac{\rho^2(b, b^{gt})}{c^2} + \alpha v$

- **Tham số:**

- α (Alpha - khác với Focal Loss): Một tham số trọng số để cân bằng.

$$IoU(A, B) < 0.5 \Rightarrow \alpha = 0$$

$$IoU(A, B) \geq 0.5 \Rightarrow \alpha = \frac{v}{(1 - IoU) + v}$$

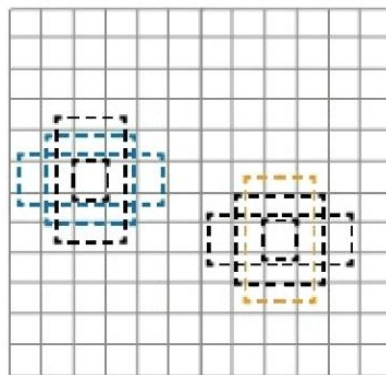
- v : Tham số đo sự nhất quán của tỷ lệ khung hình (aspect ratio). Nếu hộp dự đoán và hộp thật có cùng tỷ lệ (ví dụ cùng là hình vuông hoặc cùng hình chữ nhật đứng), v sẽ bằng 0.

7. Anchor trong Object Detection

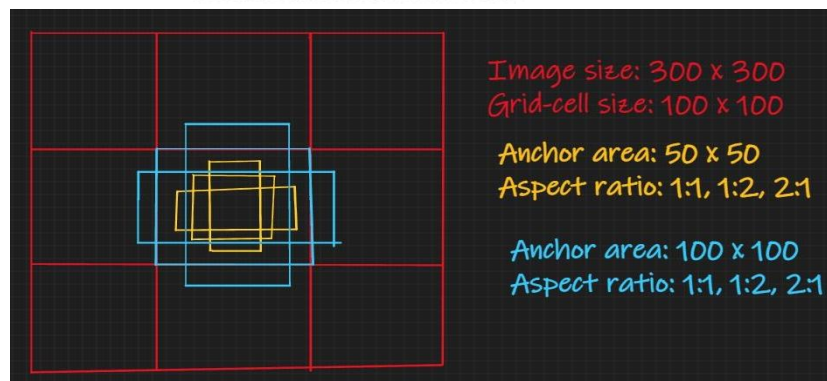
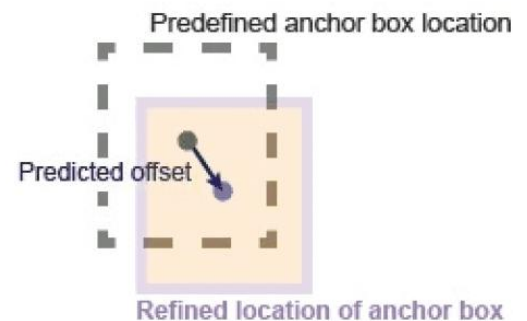
- Để phát hiện vật thể, mô hình cần biết "nên nhìn vào đâu". Đây là vai trò của Anchor (Hộp neo).
- **Định nghĩa:** Anchor là các hình hộp chữ nhật có kích thước định sẵn, được rải đều khắp bức ảnh tại các ô lưới (grid cells). Chúng đóng vai trò là "lời đề nghị ban đầu" (proposals) để mô hình dựa vào đó mà chỉnh sửa.
- **Cách hoạt động:**
 - Ảnh được chia thành lưới (grid cells)
 - Từ tâm mỗi ô lưới tạo ra các bounding boxes với kích thước/tỷ lệ khác nhau
 - Mỗi grid cell có thể chứa nhiều anchors
- Mô hình sẽ điều chỉnh các hộp này để phù hợp với các đối tượng mà nó phát hiện. Ví dụ: nếu mô hình xác định một chiếc ô tô, nó sẽ sửa đổi hộp anchor để phù hợp với vị trí và kích thước của ô tô một cách chính xác hơn.
- Mô hình sẽ học cách điều chỉnh các hộp neo (anchor) để phù hợp hơn với vị trí, kích thước và tỷ lệ khung hình của đối tượng. Điều này cho phép mô hình detect các đối tượng ở các tỷ lệ và hướng khác nhau. Tuy nhiên, việc chọn đúng bộ hộp neo có thể tốn thời gian và quá trình tinh chỉnh chúng có thể dễ xảy ra lỗi.



Ground truth image and bounding boxes



Anchor boxes at each predefined location in each feature map



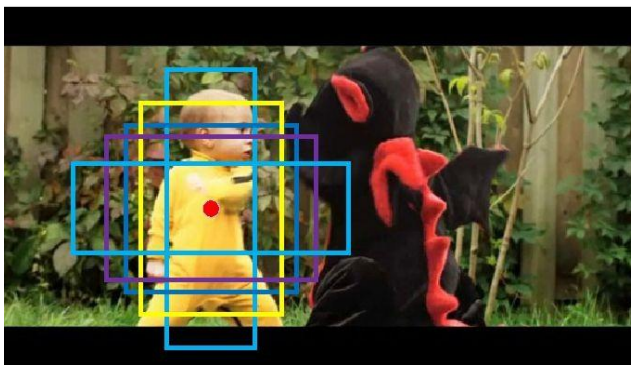
- **Phân loại phương pháp tiếp cận:**

- **Anchor-Based (Dựa trên hộp neo):**

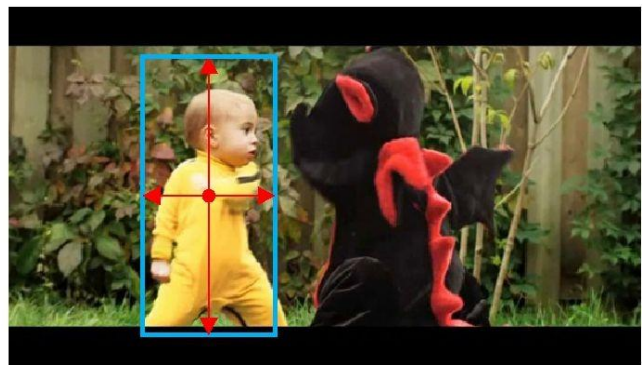
- **Định nghĩa trước các hộp mẫu** (anchors) với kích thước cố định tại mỗi ô lưới.
 - Mô hình **học cách chỉnh sửa các hộp này** để khớp với vật thể thật.
 - **Cách làm:** Dự đoán nhãn lớp và độ lệch (offset) để điều chỉnh cái hộp neo có sẵn sao cho khớp với vật thể thật. Ví dụ: Dịch cái anchor số 1 sang trái 2px, to ra 10%.
 - **Ưu điểm:** Độ chính xác thường cao do có nhiều gợi ý ban đầu.
 - **Nhược điểm:**
 - **Phức tạp:** Phải thiết kế kích thước, tỷ lệ hộp neo sao cho phù hợp với dữ liệu, cần tinh chỉnh siêu tham số phức tạp - hyper-parameter tuning.
 - **Chậm hơn:** Càng nhiều hộp neo (để bắt được nhiều vật thể) thì tính toán càng nặng.

- **Anchor-Free (Không dùng hộp neo):**

- **Dự đoán trực tiếp bounding box** (tọa độ (x, y, w, h) hoặc góc/cạnh) mà **không cần hộp mẫu**, hoặc dự đoán dựa trên các điểm đặc trưng (key-points).
 - **Cách làm:** Dự đoán trực tiếp vật thể dựa trên các điểm tham chiếu cố định (ví dụ: điểm tâm, điểm góc) mà không cần vẽ hộp trước. (Ví dụ điển hình: CenterNet ở mục 1).
 - **Ưu điểm:**
 - Thiết kế đơn giản và nhanh hơn.
 - Không cần đau đầu chọn kích thước hộp neo. Không cần tìm anchor phù hợp (hyperparameter tuning phức tạp)
 - Tốc độ suy luận (inference) thường nhanh hơn do ít anchor → kiến trúc đơn giản hơn.



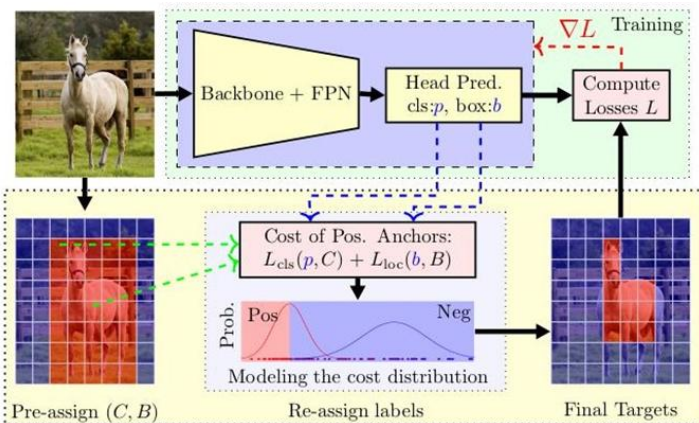
Anchor-based



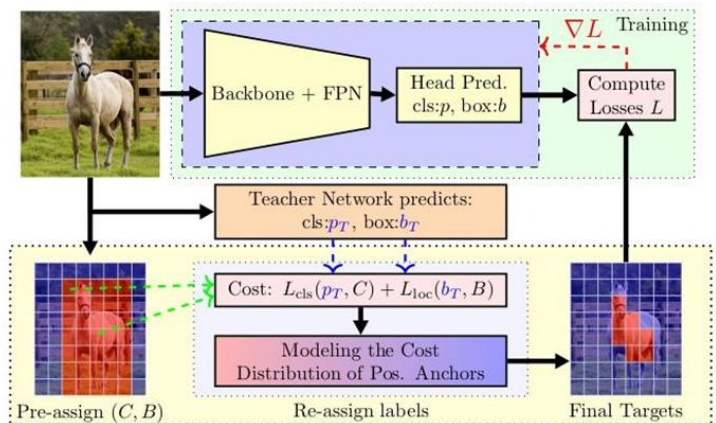
Anchor-free

8. Label Assignment - Gán nhãn

- Trong quá trình huấn luyện, làm sao mô hình biết cái hộp nào trong hàng ngàn hộp là "Đúng" (Positive - chứa vật thể) và cái nào là "Sai" (Negative - nền)? Đó là nhiệm vụ của **Label Assignment**.
- Vấn đề của phương pháp truyền thống (Conventional):** Thường dùng **IoU cố định** (ví dụ $\text{IoU} > 0.5$ là đúng). Cách này có 2 lỗi lớn:
 - Lệch pha (Misalignment):** Cái hộp có điểm phân loại cao nhất chưa chắc là cái hộp bao quanh vật thể chuẩn nhất.
 - Cứng nhắc (Rigid):** Áp đặt một luật lệ cứng nhắc cho mọi giai đoạn huấn luyện là không tối ưu.
- Các giải pháp tiên tiến (Advanced Methods):**
 - PAA (Probabilistic Anchor Assignment - Gán nhãn theo xác suất):**
 - Thay vì dùng IoU cứng nhắc, PAA tính toán một "điểm chi phí" (cost) kết hợp giữa độ chính xác phân loại và độ chuẩn xác vị trí.
 - Nó **mô hình hóa phân phối** của các mẫu dương (Positive) và âm (Negative) (thường dùng phân phối Gaussian) để **chọn ra ngưỡng tối ưu một cách tự động** và mềm dẻo.
 - LAD (Label Assignment Distillation - Chưng cất tri thức gán nhãn):**
 - Sử dụng mô hình **Teacher - Student**.
 - Một mạng "Giáo viên" (đã giỏi) sẽ dự đoán các nhãn mềm (soft labels) để hướng dẫn mạng "Học sinh".
 - Mạng học sinh sẽ dựa vào sự chỉ dẫn này để biết nên ưu tiên học mẫu nào, thay vì tự mò mẫm bằng các quy tắc IoU cứng nhắc.



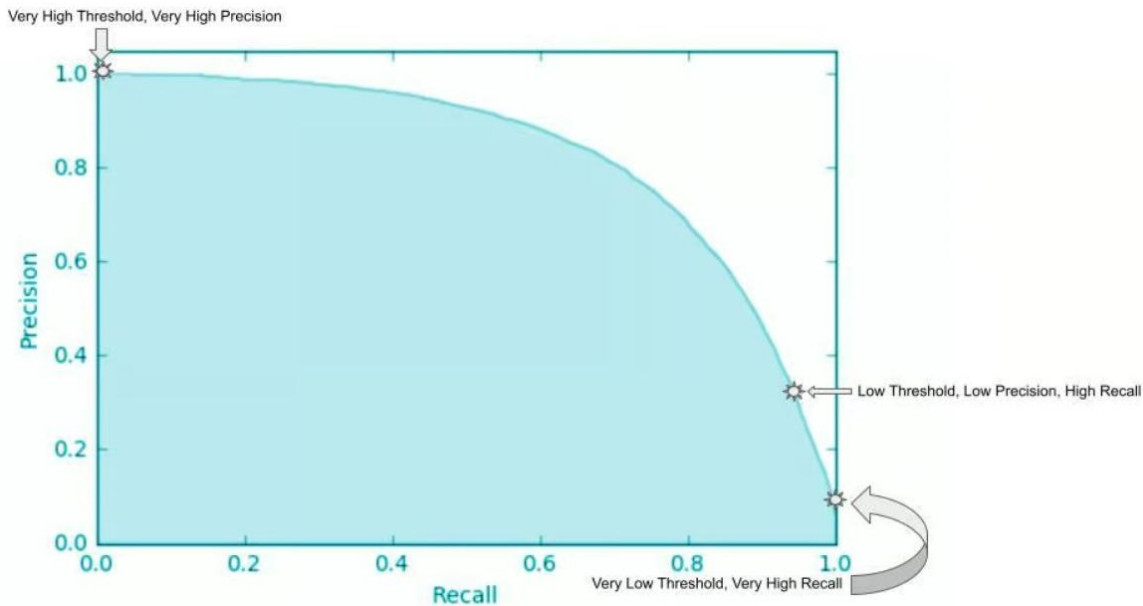
Probabilistic Anchor Assignment (PAA)



Label Assignment Distillation (LAD)

9. Metrics used for Evaluating Object Detection Models – Các chỉ số đánh giá mô hình

- Làm sao chúng ta biết mô hình "thông minh" đến mức nào? Chúng ta sử dụng các thước đo (metrics) chuẩn hóa.
- **IoU (Intersection over Union - Tỷ lệ giao/hợp)**
 - **Mục đích:** Đo xem cái hộp mô hình dự đoán (Predicted Box) trùng khớp bao nhiêu phần trăm với cái hộp thật (Ground Truth Box).
 - **Tham số:** Ngưỡng IoU (Threshold). Ví dụ, nếu $IoU > 0.5$, ta coi là phát hiện đúng. Nếu $IoU < 0.5$ coi là phát hiện sai.
- **Precision (Độ chính xác)**
 - **Mục đích:** Đo độ "tin cậy". Trong số những lần mô hình kêu "Có vật thể!", bao nhiêu lần nó nói đúng?
 - **Công thức:**
$$P = \frac{TP}{TP + FP}$$
 - **TP (True Positive):** Mô hình dự đoán ĐÚNG là có vật thể (IoU cao).
 - **FP (False Positive):** Mô hình dự đoán SAI (báo động giả, hoặc IoU thấp).
 - **TP + FP:** Tổng số lượng hộp mà mô hình đã vẽ ra (Total Prediction).
- **Recall (Độ phủ / Độ nhạy)**
 - **Mục đích:** Đo độ "bao quát". Trong số tất cả các vật thể thực tế có trong ảnh, mô hình tìm ra được bao nhiêu cái?
 - **Công thức:**
$$R = \frac{TP}{TP + FN} = TP / Total\ Ground\ Truths$$
 - **FN (False Negative):** Mô hình bỏ sót vật thể (có vật thể nhưng không báo).
 - **TP + FN:** Tổng số lượng vật thể thực tế có trong ảnh (Total Ground Truths).
- **Biểu đồ PR (Precision-Recall Curve)**
 - **Mục đích:** Xem sự đánh đổi (trade-off) giữa Precision và Recall.
 - Thường thì: Muốn phủ hết vật thể (Recall cao) thì dễ dính báo động giả (Precision thấp). Ngược lại, muốn cực kỳ chắc chắn mới báo (Precision cao) thì dễ bỏ sót (Recall thấp).
 - **Cách vẽ:** Trục tung (Y) là Precision, trục hoành (X) là Recall.
 - Đường cong càng lồi về phía góc trên bên phải (1,1) thì mô hình càng tốt.



- **AP (Average Precision - Độ chính xác trung bình)**

- **Mục đích:** Một con số duy nhất đại diện cho chất lượng của mô hình đối với một lớp cụ thể (ví dụ: lớp "chó").
- **Ý nghĩa công thức:** AP không phải là trung bình cộng đơn thuần của Precision. Nó chính là **Diện tích dưới đường cong PR (Area Under the PR Curve)**.
- **Tại sao dùng AP?** Nó tổng hợp được hiệu suất của mô hình ở mọi ngưỡng độ tin cậy khác nhau.

- **mAP (Mean Average Precision)**

- **Mục đích:** Thước đo tổng quát nhất cho **toàn bộ mô hình** (đa lớp).
- **Công thức:** $mAP = \frac{1}{n} \sum AP$ → Trung bình AP trên tất cả các class.
 - **n:** Số lượng lớp (class) cần phát hiện (ví dụ: chó, mèo, xe hơi... thì n=3).
 - **AP:** Độ chính xác trung bình của từng lớp.
- **Ý nghĩa:** Nó là **trung bình cộng của AP tất cả các lớp**.
- Ví dụ: AP(chó) = 0.8, AP(mèo) = 0.6 → mAP = 0.7.

Lưu ý quan trọng về IoU Threshold:

- Giá trị Precision và Recall phụ thuộc hoàn toàn vào việc bạn chọn ngưỡng IoU là bao nhiêu (ví dụ 0.5 hay 0.75).
- Vì vậy, khi báo cáo kết quả, người ta thường ghi rõ:
 - **mAP@0.5** (mAP tính tại ngưỡng IoU=0.5)
 - **mAP@[0.5:0.95]** (trung bình mAP chạy từ ngưỡng 0.5 đến 0.95).

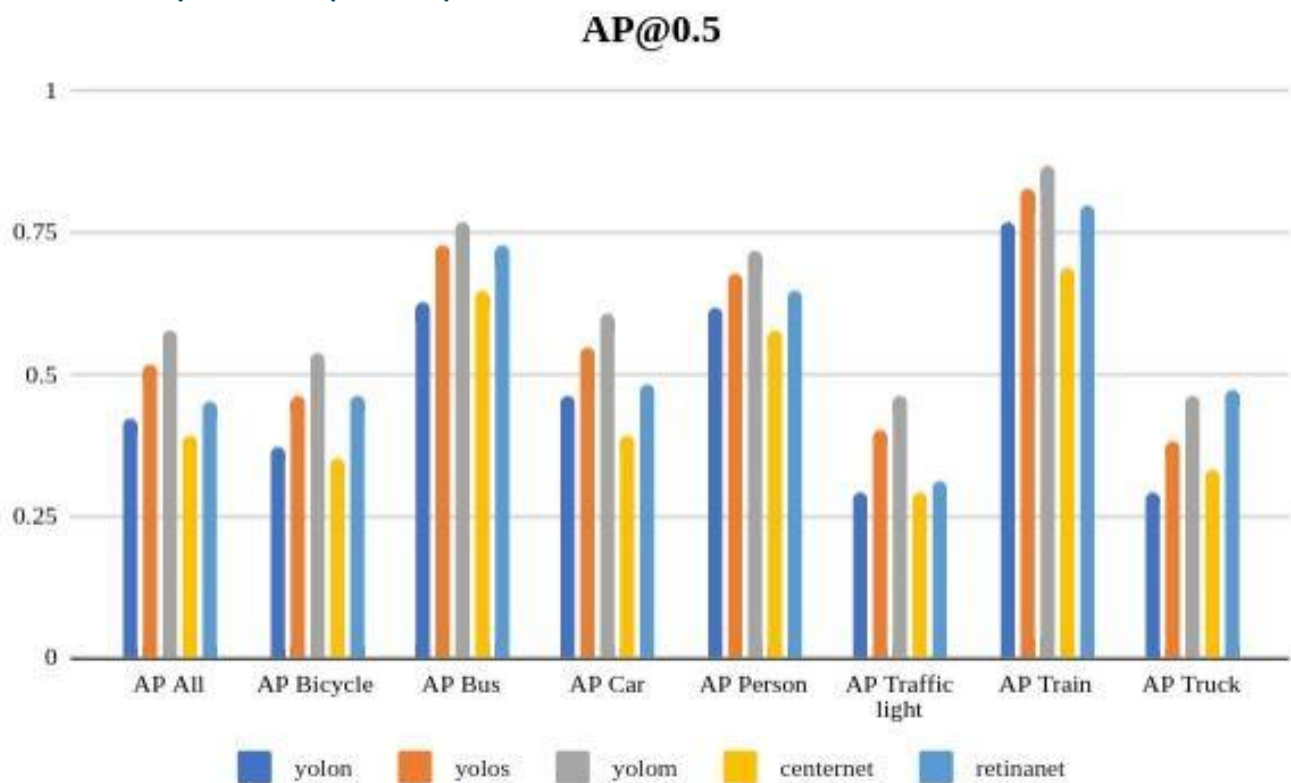
10. Những thách thức chính của Object Detection (Challenges)

Để xây dựng một hệ thống nhận diện tốt, ta phải vượt qua các rào cản sau:

- **Quy mô và Độ phức tạp (Scale & Complexity):** Thế giới có vô vàn vật thể với hình dáng đa dạng.
- **Xử lý thời gian thực (Real-time Processing):** Xe tự lái cần nhận diện ngay lập tức, không thể chờ đợi. Cân bằng giữa nhanh và chính xác là rất khó.
- **Dữ liệu lớn (Large-scale Datasets):** Cần hàng triệu ảnh có gán nhãn để dạy mô hình, việc này tốn kém và mất thời gian.
- **Khả năng thích nghi (Adaptability):** Mô hình học ở Mỹ có thể nhận diện sai biển báo ở Việt Nam. Nó cần thích nghi với môi trường mới mà không cần học lại từ đầu.

11. So sánh Hiệu năng và Tốc độ thực tế

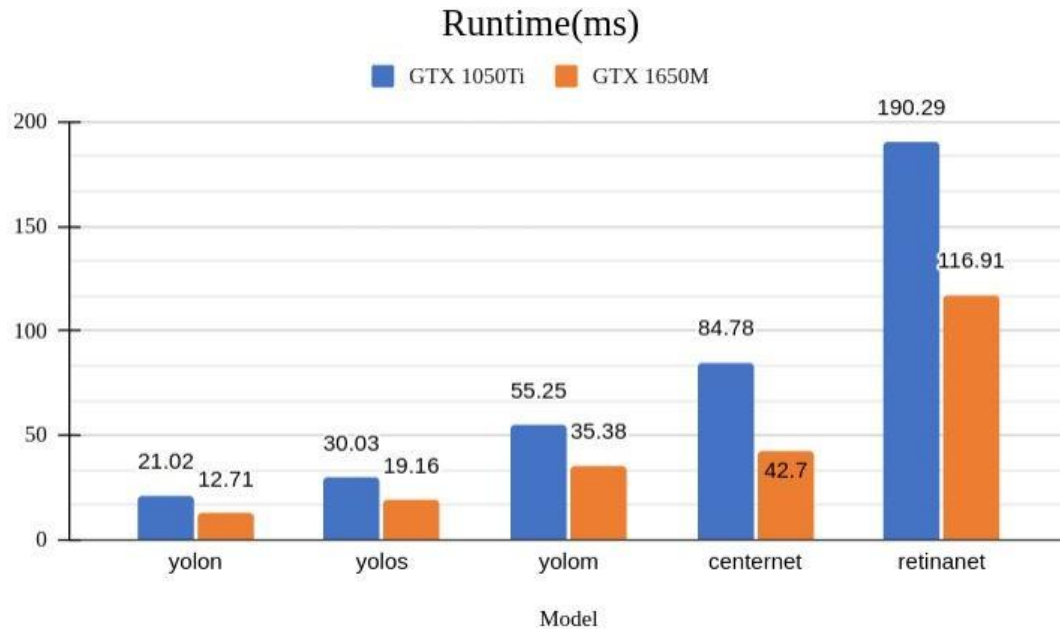
11.1. So sánh về Độ chính xác (AP@0.5)



Biểu đồ cho thấy **độ chính xác trên các lớp vật thể khác nhau** (Xe đạp, xe buýt, người...):

- **RetinaNet (Màu xanh dương nhạt):** Thường đạt độ chính xác cao nhất hoặc nhì trong hầu hết các lớp (đặc biệt là xe buýt, tàu hỏa). Điều này nhờ vào thiết kế Focal Loss và FPN.
- **YOLO-m (YOLO medium - Màu xám):** Rất cạnh tranh, đôi khi vượt qua cả RetinaNet (như ở lớp Xe hơi, Người).
- **YOLO-n (YOLO nano - Màu xanh đậm):** Phiên bản nhỏ nhất, chấp nhận độ chính xác thấp hơn để đổi lấy tốc độ.
- **Mô hình chính xác nhất:** YOLOm, RetinaNet
- **Thứ tự chung:** YOLO series (tùy version) – RetinaNet – CenterNet – SSD

11.2. So sánh về Tốc độ (Runtime - ms)



Thời gian xử lý trên một bức ảnh (thấp hơn là tốt hơn):

- **YOLO-n (nano):** "Vua tốc độ". Chỉ mất ~12ms trên GPU 1650M và ~21ms trên 1050Ti. Phù hợp cho điện thoại, thiết bị nhúng.
- **RetinaNet:** Chậm nhất trong nhóm so sánh (~116ms - 190ms). Chậm hơn YOLO-n gần 10 lần.
- **CenterNet:** Nằm ở giữa (~42ms - 84ms), cân bằng khá tốt.
- **Nhanh nhất:** YOLO (12-21ms)
- **Chậm nhất:** RetinaNet (117-190ms)
- **Thứ tự tốc độ:**
 1. YOLO (~12-21ms) ⚡
 2. YOLOv5 (~19-30ms)
 3. SSD (~35-55ms)
 4. CenterNet (~42-84ms)
 5. RetinaNet (~117-190ms)

11.3. Kết luận chọn mô hình

- **Cần Real-time (Camera giao thông, Drone):** Chọn **YOLO (v5, v8...)**. Tốc độ xử lý siêu nhanh bù đắp cho một chút thiệt thòi về độ chính xác.
- **Cần Chính xác tuyệt đối (Y tế, Soi lỗi sản phẩm):** Có thể cân nhắc **RetinaNet** hoặc các dòng **YOLO size lớn (L, X)**.

Collaborator: Nguyễn Lê Hiếu Nhi**Danh mục tài liệu sử dụng:**

- **Tài liệu học phần:** TS. Nguyễn Thị Khánh Tiên, *Chapter 6. Image Recognition - Topic: Object detection survey, Slide Học phần Xử lý ảnh & Thị giác máy tính*, Viện Công Nghệ Thông Tin, Điện & Điện Tử, Trường Đại Học Giao Thông Vận Tải Tp.HCM.
- **Tài liệu tham khảo:**
 - Trần Lê Anh (5/2025), *Chapter 5: 2D Object Detection, Slide Image Processing and Computer Vision*, Viện Công Nghệ Thông Tin, Điện & Điện Tử, Trường Đại Học Giao Thông Vận Tải Tp.HCM.
 - <http://ultralytics.com/vi/blog/benefits-ultralytics-yolo11-being-anchor-free-detector>
 - Một số ảnh trên Internet.
 - Các công cụ AI hỗ trợ.